Task Engine

A State-Aware Execution Architecture for Desktop Automation

Author:
Arnav Sachdeva
CSE 2028
PEC Chandigarh
sachdevaar9@gmail.com

Date: March 2026

Overview

This document presents the design of a task execution system for desktop environments.

Traditional systems operate through **deterministic mappings** from user intent to predefined actions. While this approach provides reliability and predictability, it is inherently limited to fixed command sets and **lacks the ability to plan** and execute multi-step goals.

In contrast, **modern agent-based systems** generate action sequences dynamically, enabling **more complex, multi-step tasks**. However, their internal decision-making is typically **opaque, non-deterministic, and difficult to verify**. This introduces uncertainty in intermediate steps, including incorrect assumptions and unpredictable behavior, making them unsuitable for desktop environments where correctness and predictability are essential.

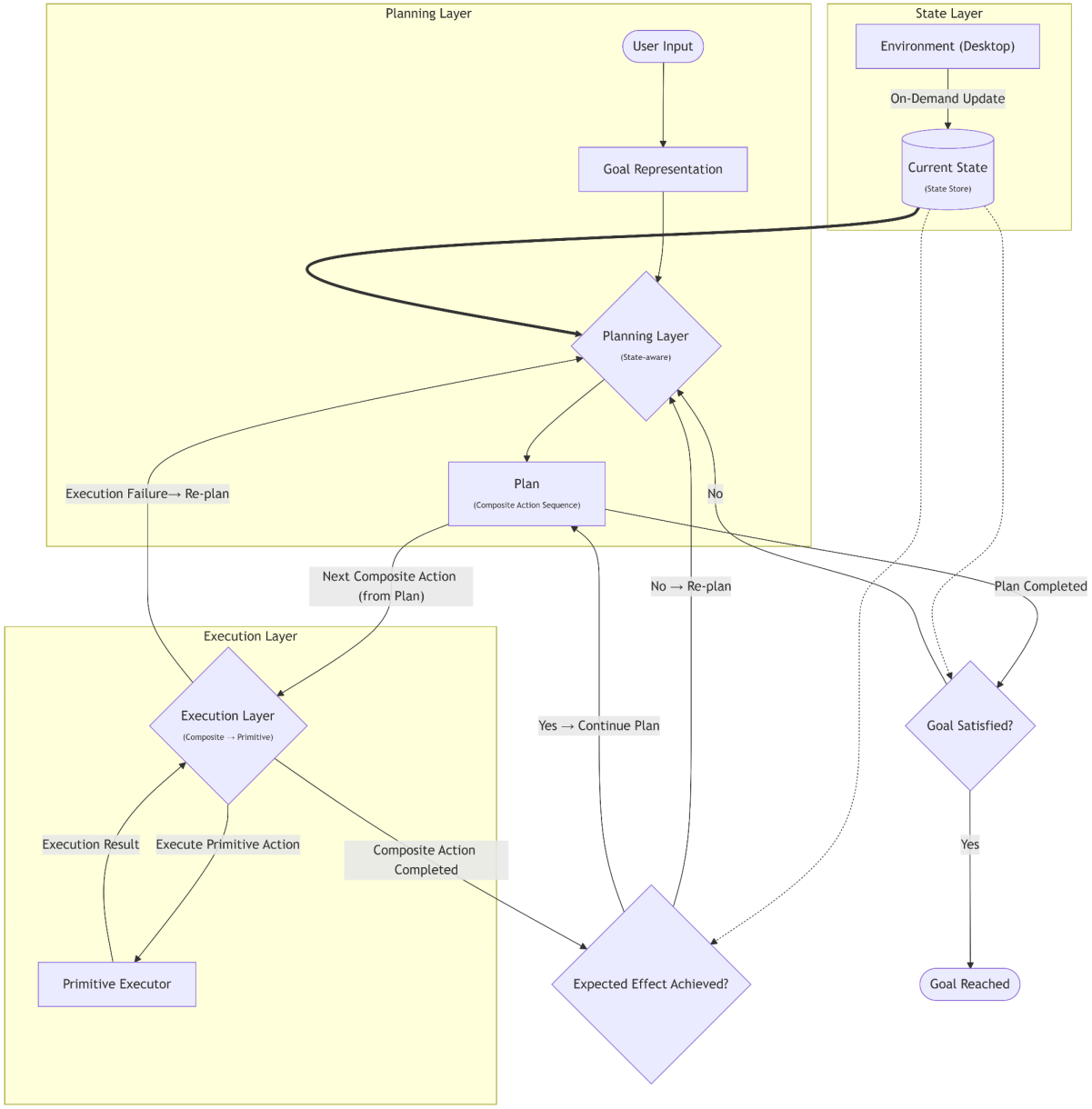
As a result, **reliability and flexibility** are effectively achieved by **two separate classes** of systems. There is currently **no widely adopted approach that combines structured planning with reliable and inspectable execution.**

This work is motivated by the need for a system that explicitly represents goals, decomposes them into actionable steps, and executes them in a controlled and verifiable manner.

To address these limitations, the system is designed as an **engine that introduces structured planning over desktop interactions.**

This enables predictable behavior while preserving the ability to handle multi-step, context-dependent tasks.

Architecture



Closed-loop, state-aware task execution architecture with hierarchical failure handling and deterministic control
Solid arrows: Control flow
Dotted arrows: State dependency
Thick arrows: Strong dependency (state → planner)

Outline

This architecture defines a **state-aware, goal-oriented execution engine** that operates within a **closed feedback loop**. It separates decision-making from action handling via a planning layer that generates structured action sequences based on the current environment state, and an execution layer that performs them via predictable, low-level steps.

At a high level, it **takes a goal representation, produces a plan** consisting of composite actions, executes them step-by-step using primitive actions, and continuously **validates outcomes against the real environment**. The system does not assume state transitions; instead, it relies on **on-demand observation** to ensure that execution remains grounded in real desktop conditions.

State Layer

The system maintains an explicit representation of the current environment state, referred to as the state store. This state **captures relevant aspects of the environment context**, such as active windows, the currently focused window, and UI structures.

State observation is primarily performed through OS-level metadata and accessibility interfaces, with fallback mechanisms using OCR and computer vision techniques when structured UI information is not accessible.

Current State (State Store) is **updated on-demand during validation and planning** via direct perception of the (Desktop) environment, ensuring that decisions are based on the actual state rather than inferred outcomes of previously executed steps.

This layer acts as the **single source of truth** for the system. It is used by the planning layer to determine applicable actions and by the validation process to verify whether the expected effects and final goals have been achieved.

Planning Layer

This layer is **responsible for deriving a plan** to achieve the user-specified ‘Goal State’ from the presently inferred ‘Current State’. It **selects and orders the ‘Composite Actions’** required to achieve a given goal.

The output plan contains a sequence of ‘Composite Actions’. *Each Composite Action represents a meaningful unit of work with defined pre-State and post-State*. The planner constructs valid sequences by *chaining actions through pre-condition and post-condition compatibility.*

In addition to initial plan generation, *the planning layer is invoked whenever the observed effect deviates from expectations.* **Upon failure** or mismatch between expected and observed outcomes, the planner re-evaluates the current state and **produces an updated plan**. This ensures that it can recover from unexpected conditions while maintaining structured and interpretable behavior.

The planning layer is responsible only for selecting and ordering actions; it does not perform execution. Execution is handled by the subsequent layer.

The planning approach is **conceptually aligned** with **classical formalisms** such as **STRIPS**, where actions are defined through preconditions and postconditions and composed based on **state compatibility**.

However, unlike classical planners operating over fully known symbolic state spaces, this system performs **state-driven recomposition** under **partial observability**, relying on **real-time state acquisition** rather than exhaustive search over a predefined graph.

Abstract Structure of A Composite Action

Execution Layer

This layer is **responsible for executing the composite actions** produced by the planner. Each *composite action* is decomposed into a *sequence of primitive actions*, which are executed through direct interaction with the operating system.

Primitive actions represent the lowest level of control, such as input simulations, GUI interactions, and file system searches. These are executed sequentially, and their outcomes are reported back to this layer.

This layer **handles local control logic**, including retries and minor fallbacks for low-level failures. However, it doesn't alter the overall plan. If the intended effect of a composite action is not achieved, control is returned to the planning layer for re-evaluation.

Primitive sequences are not resumed after interruption; instead, failures trigger retry or re-evaluation. Where possible, primitives are designed to be safely repeatable.

Control Flow and Validation Loop

Execution proceeds as a **closed-loop process** that alternates between planning, execution and validation.

After each composite action is executed, it **performs validation** by comparing the current state **with its expected 'post-State'**. This validation is grounded in real observations rather than assumed transitions.

If the expected effect is **achieved**, execution **proceeds to the next** composite action. If the validation **fails**, it **triggers re-planning** using the updated current state.

Additionally, critical failures result in immediate re-planning without attempting to continue the current plan.

After performing the complete plan, the system performs final validation to determine whether the goal state has been satisfied. If not, it re-enters the planning phase – with ‘current state’ as initial and same goal state.

Failure Handling Strategy

Failure handling in this architecture is structured across two levels.

At the **primitive action level**, failures are **handled** locally **within the execution layer** through retries and minor corrective mechanisms. These are intended to address transient issues without involving the planner.

At the **composite action level**, failure is defined as the inability to achieve the ‘post-State’ on the (desktop) environment. Such failures **trigger re-planning**, where the planning layer re-evaluates the current state and generates an alternative plan.

This hierarchical approach ensures that **low-level instability** is **managed effectively**, while **higher-level inconsistencies** are **resolved through deliberate decision-making** based on current state.

Illustrative Example

The following example illustrates how the architecture handles a multi-step task in a real desktop environment:

Upload the file “CN Assignment 3” to Assignment 3 in the Networks course on Google Classroom.

Goal State Representation

The goal is expressed as a set of verifiable conditions that must hold in the environment:

```
{
 "goal_conditions": [
  "platform_active(Google_Classroom)",
  "course_open(Networks)",
  "assignment_open(Assignment_3)",
  "file_uploaded(CN_Assignment_3, Assignment_3)"
 ]
}
```

Plan (Composite Actions)

Based on the current state, the planner generates the following sequence:

1. Locate_File("CN Assignment 3")
2. Open_Assignment("Google Classroom", "Networks", "Assignment 3")
3. Upload_File("CN Assignment 3", "Assignment 3")

Composite Action

Each step represents a composite action that transitions the system closer to the desired goal state.

Example Composite Action
Composite Action: Upload_File

Preconditions:

assignment_open("Assignment 3")
file_path("CN Assignment 3") is available

Postconditions:

file_uploaded("CN Assignment 3", "Assignment 3")

Primitive Action Breakdown (Execution Layer)

The composite action is executed through a sequence of primitive actions:

1. Focus browser window
2. Locate upload interface
3. Trigger file selection dialog
4. Provide file path
5. Confirm upload
6. Wait for upload completion signal

These primitives represent direct interactions with the environment and are executed sequentially, with feedback used to ensure correct completion.

The Recovery Branch: Handling Environmental Entropy

When the **primary automation layer (UIA)** encounters an **information-deficient** state—common in browsers where internal elements are not exposed to the OS—the **engine initiates** an **automated recovery branch**.

This **ensures the plan remains viable** by transitioning from Object-Metadata to Pixel-Level Verification.

Phase	Event / Observation	Architectural Recovery Response
Detection	**UIA Probe Timeout:** The backend fails to resolve a unique handle for the "Add or Create" element within the process window.	**Java Engine** halts the linear instruction and promotes the task to a **Vision-Required State**.
Diagnosis	**Environment Scan:** Python invokes the CV/OCR atom to analyze the active viewport.	**Feature Extraction:** The system performs **Template Matching** to identify the visual signature of the target button, bypassing the lack of structural metadata.
Correction	**Dynamic Coordinate Injection:** The Planner generates a localized **Spatial Atom** based on the visual hit.	**Action:** `click_at(x, y)` is executed, using the visually-acquired anchor to bridge the gap left by the opaque UI layer.
Resumption	**Stabilization Check:** `ui_stabilized: true`.	**Contextual Resumption:** The high-level plan resumes from the current coordinate, treating the visual confirmation as a verified state-change.

Validation and Feedback

After execution, the system verifies whether the expected effect has been achieved by checking the current environment state.

If the condition $\text{file\_uploaded}(\text{CN\_Assignment\_3}, \text{Assignment3})$ is satisfied, the system proceeds. Otherwise, it triggers re-planning based on the updated state.

This architecture exhibits the following key characteristics that define its behavior and operational properties.

Design Characteristics

- **State is observed, not assumed,** ensuring alignment with real environment conditions.
- The architecture **cleanly distinguishes** between **decision-making** (planning layer) and **plan realization** (execution layer), preventing intermixing of logic.
- **Planning** is **explicit and interpretable**, based on structured control flow representations.
- **Primitive actions** are executed in a **controlled and predictable** manner, avoiding non-deterministic or black-box behavior.
- Validation occurs after each composite action, enabling **early failure-detection and recovery.**
- **Recovery** is handled **through state-driven re-planning**, not implicit correction.
- The architecture follows a **closed-loop control structure** – continuously cycles between planning, execution and validation, allowing it to **adapt dynamically** to real-world conditions.
- **Low-level failures** are **handled locally** within the execution layer, while **higher-level failures trigger re-planning**, ensuring structured recovery – creating a ***Hierarchical failure handling system***.

A critical aspect of this design is the abstraction of actions, which enables the separation between planning and execution.

Action Abstraction

This system introduces a **two-level action abstraction** to bridge **high-level planning** with **low-level steps**.

At the lower level, **primitive actions** correspond to **direct interactions with the operating system**, such as input simulation, GUI interactions, or file system access. These actions are **consistent, executable units** that affect the environment without carrying semantic meaning beyond their immediate effect. They are designed to be **minimal, reusable, and environment-facing**.

At the higher level, **composite actions** represent **meaningful units of work** defined in terms of their expected effect on the environment state. Each composite action is **characterized by a pre-state and a post-state**, allowing it to be reasoned about within the planning process.

This abstraction enables the **planning layer** to **operate over semantically meaningful actions**, while the **execution layer handles the procedural details** required to realize them. **By separating** what needs to be achieved from how it is executed, the system maintains **clarity in decision-making** while retaining **control over operation**. *Primitive actions are intentionally constrained to remain minimal and predictable to avoid introducing ambiguity into workflow.*

Additionally, defining actions in terms of state transitions allows composite actions to be **reusable and composable**, enabling the planner to **construct valid action sequences based on state compatibility** rather than fixed command mappings.

This design is conceptually aligned with ***hierarchical task planning approaches***, where high-level actions are decomposed into executable steps.

While the above sections describe the intended system design, the current implementation reflects an earlier stage in this evolution.

Implementation Snapshot

Core Philosophy

The architecture establishes a strict **separation of concerns** between **orchestration and execution**. The task engine is engineered as a **Java-first system**, utilizing **Python** as a **bounded native automation backend**. This division ensures **deterministic control flow** and **failure isolation** while leveraging platform-native capabilities.

Java: Orchestration Layer (Control Plane)

Java serves as the **central runtime** for the engine, responsible for the **high-level logic** and **state-driven workflows**:

- **Task Parsing & Decomposition**: Breaking user goals into actionable sequences.
- **Execution Context**: Maintaining the "State Store" and managing multi-step transitions.
- **Recovery Logic**: Owning all retry policies, fallbacks, and branching decisions.
- **IPC Management**: Orchestrating the lifecycle of the execution backend.

Java provides the **formal structure for the control plane**, leveraging **strong typing** and **deterministic control flow** to implement the engine's core logic, state-aware workflows, and hierarchical recovery mechanisms.

Python: Execution Backend (Data Plane)

Python is utilized exclusively as a **specialized capability provider** for **Windows UI Automation** (via pywinauto) and **perception** tasks (OpenCV/OCR):

- **OS-Level Interaction**: Window management, focus activation, and UI tree inspection.
- **Input Injection**: Low-level mouse and keyboard event simulation.
- **Bounded Scope**: Python is restricted from making orchestration or branching decisions.

Each invocation is designed to be **atomic, deterministic, and side-effect focused**.

High-Performance Native Primitives (Java NIO)

While the UI layer is handled via the Python backend, the engine integrates a **high-performance Intent-Based File Search module** natively in Java.

- **Mechanism**: Built using the **Java NIO (Files.walkFileTree) API**, the module performs **recursive, non-blocking filesystem traversals** to resolve file-path dependencies.
- **Logic**: It utilizes **Regex-driven fuzzy matching** to map high-level user intents (e.g., "CN Assignment 3") to absolute system paths.
- **Architectural Role**: This serves as a **Native Primitive Action**. By keeping filesystem heavy-lifting within the JVM, the system **avoids** the **overhead of inter-process communication for data-intensive I/O operations**, ensuring near-instantaneous state resolution for file-related goals.

IPC & Execution Boundary

The system enforces a strict **process-level boundary** to isolate the orchestration logic from the volatility of desktop automation:

- **Deployment & Packaging**: To guarantee **environment consistency** and **zero-dependency execution**, the Python backend is architected to be distributed as a **frozen native binary** (via PyInstaller/Nuitka). This eliminates "it works on my machine" issues by **bundling the entire automation runtime**.
- **Communication Protocol**: Java interfaces with the backend via ProcessBuilder, leveraging a **high-performance JSON-based IPC** over the **process input-output streams**. This allows for **structured telemetry** and state-reporting **without** the overhead of **network sockets**.
- **Failure Isolation**: This **decoupled model** ensures that any crashes, memory leaks, or hangs within the UI automation layer are **physically isolated** from the core Java engine, **preserving the integrity of the state-machine and recovery logic.**

Atomic Task Model

All interactions are modeled as **atomic units**, which Java **composes into higher-level workflows**.

- **Current Library Includes**: focus_window, click, list_windows, query_element_state.
- **Design Contracts**: Each task is built for **single responsibility** and **explicit success/failure signaling**, providing the necessary telemetry for the **closed-loop feedback system**.

Execution & Recovery Strategy

The engine supports a **hybrid execution model** to balance performance with safety:

- **Single-Action Invocation**: Maximum isolation for sensitive state changes.
- **Centralized Recovery**: Python reports structured error codes (e.g., ELEMENT_NOT_FOUND); Java interprets these to trigger hierarchical recovery (Retry → Re-sync → Re-plan).

Summary

Control Plane: Java (Core Engine) → Owns the logic, state, and goal-orchestration.
Execution Plane: Python → Owns the native UI interaction and perception.
Java → Selective high performance native primitives (intent based file search).

The resulting architecture **transforms** desktop automation from **a collection of fragile, state-agnostic scripts into a robust, goal-conditioned orchestration engine.**

System Interoperability & Data Contracts

The system uses a **process-isolated**, **bidirectional communication** bridge between the Java orchestration layer and the Python execution backend.

Communication is implemented over **standard input/output streams** using a **length-prefixed JSON protocol**.

Protocol Design

- Each message is **prefixed with a 4-byte length header** followed by a JSON payload.
- Ensures complete message reads and **prevents stream fragmentation.**
- Enables **deterministic parsing** without partial or corrupted data.

Interaction Model

- **Java** → sends structured **execution commands**
- **Python** → executes atomic actions and returns **structured telemetry**

This establishes a strict **request–response contract**:
- No shared state across processes
- All outcomes are explicitly reported
- Failures are surfaced as structured error responses

Minimal IPC Handling (Java side)

Sending Command from Java (Core Engine) to Python process

```
// Send command
public void send(Object obj) throws Exception {
    byte[] data = mapper.writeValueAsBytes(obj);
    out.writeInt(data.length);
    out.write(data);
    out.flush();
}
```

Receiving response from Python process back to Java (Core Engine)

```
// Receiving response
    public JsonNode recv() throws Exception {
        int length = in.readInt();
        byte[] buf = new byte[length];
        in.readFully(buf);
        return mapper.readTree(buf);
    }
```

This design ensures **strong isolation** between **orchestration and execution**, while maintaining **low-overhead**, **reliable communication** suitable for real-time desktop interaction workflows.

Resource, Latency & Reliability Metrics

The system is designed to **balance execution isolation with acceptable interaction latency**.

Each composite action is executed in a **fresh process**, ensuring a **clean runtime** at the cost of a bounded invocation overhead.

- **Invocation Latency Overhead**
  Each process invocation introduces the following overhead:

Step	Latency (ms)	Notes
Python interpreter startup	10–20 ms	Low for CPython on local system
Module imports (`pywinauto`, `opencv`, OCR engine)	30–70 ms	Depends on Python version and disk speed
Backend initialization (UIA / Win32)	5–10 ms	Establish connection to system automation layer
Total per process invocation	~50–100 ms	First primitive action after spawn pays this overhead

- **Resource Footprint**:

Component	CPU	RAM	Notes
Python interpreter + modules	2–5%	50–100 MB	Idle after init
pywinauto / UIA backend	negligible	<10 MB	OS API calls
CV + OCR	10–30% per OCR call	50–150 MB	Occasional, only per fallback primitive
Coordinate fallback	negligible	<1 MB	Optional

- **Primitive Action Execution Latency**
  Once the backend is initialized, individual primitives operate at near-native speeds:

Primitive	Latency per Primitive	Notes
UIA / pywinauto primitive	5–10 ms	Includes first-time `WindowSpecification` lookup
CV template match (Localized Region of Interest (ROI))	30–50 ms	Screenshot + preprocessing + template matching
OCR (Localized ROI)	50–100 ms	Tesseract / Windows OCR first-time initialization included
Coordinate fallback	5–10 ms	OS-level mouse/keyboard, negligible

Key Observations

- Process **isolation** introduces **~50–100 ms overhead per composite action**.
- **Primitive execution** remains **low-latency**; overall delay is dominated by UI response time.
- The **added overhead** remains **acceptable for composite desktop interactions**, where end-to-end **latency** is **primarily** governed **by application response** rather than execution cost.
- **Tradeoff**: isolation ↑ vs latency ↑ → **isolation chosen for reliability**.
- **Resource usage remains manageable** on standard laptops.

File Search Performance

The **Java-native file search** module (NIO-based traversal) completes a **full filesystem scan in ~10 seconds** for a typical student environment, **avoiding IPC overhead** for data-intensive operations.

Current System and Ongoing Development

The current implementation establishes a desktop-level execution engine capable of interpreting user inputs and translating them into **structured sequences of system-level actions**, including application control, file search and environment interaction.

Execution is handled through a *set of low-level modules* that interact directly with the operating system, enabling actions such as input simulation, window manipulation, and file system scanning. These components form a **reliable foundation for the primitive action layer**.

Additionally, a **rule-based intent parsing mechanism** is partially integrated, allowing extraction of structured parameters from user queries **in specific domains such as file search**. This enables more **flexible and context-aware behaviour** compared to static command mappings.

While the **core execution pipeline** is **operational**, several components are **being actively evolved** toward the proposed architecture. Execution is currently driven by direct input-to-action mappings, with **higher-level planning being extended** to support structured multi-step sequences of composite actions.

High-level behaviors that currently exist as **procedural logic** are being **abstracted into Composite Actions**. These are **defined by explicit Pre-State and Post-State conditions**, enabling structured and reliable multi-step execution.

State handling is similarly **transitioning** from localized observation **toward a centralized, queryable state representation**, enabling consistent state-based reasoning. In parallel, the **control flow** is **being extended to** incorporate **a formal validation loop and structured re-planning**, progressively shifting failure handling from local correction toward state-driven recovery.

The ongoing development focuses on *restructuring the system from direct input-to-action mappings toward a state-driven architecture* with explicit goal representation, structured action abstractions, and adaptive multi-step planning.

This transition **establishes a shift** from direct command handling **to structured, state-driven task orchestration.**